

[Open in app](#)[Get started](#)

Shahriar Hossain

[Follow](#)Jun 30 · 7 min read · [Listen](#)

Save



My Selenium Journey

By Shahriar Hossain

What do I want to do?

- I was working on a project. The project's main theme is, User will submit their idea in competition, and after voting the winner will be selected.
- My task here is to automate the idea submission of this project.

What do I need?

- Here at first, I need to explore the system and figure out how the idea submission works. I needed a good configuration laptop so that I could run the script. After that, I need to figure out what automation tool I will use here.

What is my plan?

- I plan to use selenium as here we will do UI testing and Selenium is one of the best here.

What is selenium?

- Selenium is a free (open-source) automated testing framework to validate web applications across different browsers and platforms. (Copy paste from the internet.)



[Open in app](#)[Get started](#)

Selenium

What we need before start using Selenium.

- First of all, it is required to identify tasks that an application has to accomplish.
- Second, a set of necessary input data has to be created.
- Third, expected results have to be defined in order for one can judge that an application (a requested feature) works correspondingly.
- Fourth, Execute a test.
- Finally, Compare expected results with actual results, and decide whether the test has been passed successfully.

Selenium setup

- Selenium Webdriver (Supports all major browsers, we use Mozilla, chrome, and IE)
- IntelliJ IDEA
- Java (JDK)
- TestNG & JUnit is integrated with the IntelliJ Idea libraries.
- Reference link: [*Selenium Setup in IntelliJ*](#)

IntelliJ IDEA



[Open in app](#)[Get started](#)

JUnit are there in the libraries.

Webdriver, ChromeDriver, setProperty

- **WebDriver** is an open-source tool for automated testing of web apps across many browsers. It provides capabilities for navigating web pages, user input, JavaScript execution, and more. **ChromeDriver** is a standalone server that implements the W3C WebDriver standard.
- WebDriver is like a data type. We use data types like int and float for a variable. Here for selenium, we use WebDriver as a data type.

```
WebDriver driver;
```

And for running the test in the google chrome browser we use ChromeDriver.

```
driver = new ChromeDriver();
```

For other browsers like firefox or internet explorer, we have to use other drivers.

```
driver = new FirefoxDriver();
```

```
driver = new InternetExplorerDriver();
```

Sometimes for the update of the browser, this code is not working. In that case, we need to use the **setProperty**. Where we need to download the chrome driver or gecko driver & let the code know where the driver is. And that time you do not need to use the above lines.

Here at first, I mentioned which browser or driver I want to get then I mentioned the local directory where it is.



[Open in app](#)[Get started](#)

```
driver.get("URL");
```

To open a website you need to provide the URL to the driver.get().

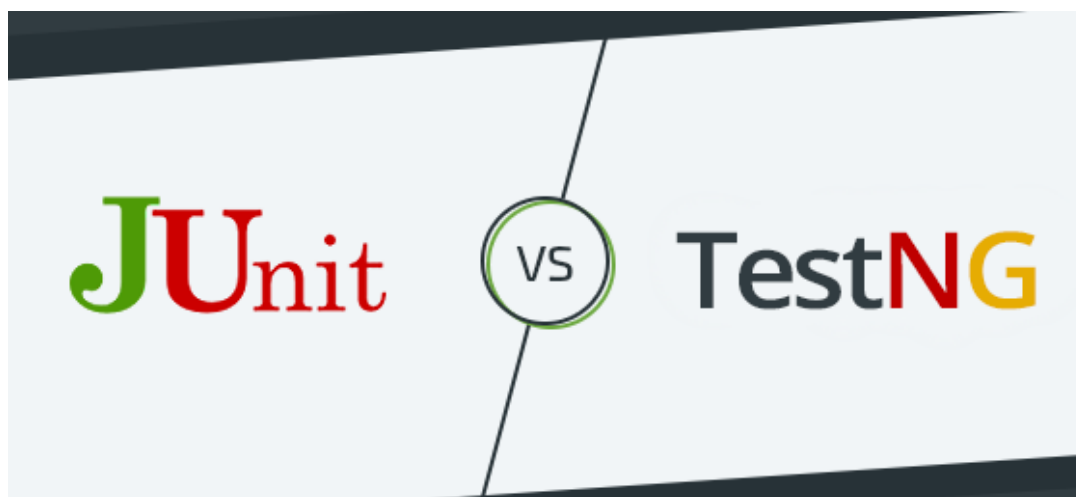
Chropath, XPath finder, find elements.

- After opening the website then the real work will start. Now we need to work on each element.
- We need to inspect all the elements to find their id, name, CSS selector, className, tagName, etc. I mostly use XPath or id. To find those sometimes I face difficulties. To find those easily I use some extensions called Chropath, and Xpath finder.
- **Chropath** link: [Chropath](#)
- **Xpath Finder** Link: [XpathFinder](#)

Programming languages. (Java, JS, Python, Ruby, C#)

- Selenium supports JAVA, Javascript, Python, Ruby, and C#. It supports object-oriented languages. I use **Java** for my script.

Junit, TestNG



[Open in app](#)[Get started](#)

- We use **TestNG** or **JUnit** for creating test cases. We must have a lot of test cases while creating a script. We need to differentiate them from each other. We can do this by using TestNG or JUnit.
- Details about TestNG and JUnit: [TestNG & JUnit](#)
- Test Annotations (BeforeTest, AfterTest, Test)
- Both JUnit and TestNG use annotations and almost all the annotations look similar. TestNG uses @BeforeMethod, @AfterMethod similar to @Before, @After in JUnit4.



[Open in app](#)[Get started](#)

Execute before the first test method in the current class is invoked.	<code>@BeforeAll</code>	<code>@BeforeClass</code>
Execute after all the test methods in the current class have been run.	<code>@AfterAll</code>	<code>@AfterClass</code>
Execute before each test method	<code>@BeforeEach</code>	<code>@BeforeMethod</code>
Execute after each test method	<code>@AfterEach</code>	<code>@AfterMethod</code>
Ignore Test	<code>@ignore</code>	<code>@Test(*enable=false*)</code>
Expected exception	<code>@Test(*expected=ArithmeticException.class*)</code>	<code>@Test(*expectedException=ArithmeticException.class*)</code>
Timeout	<code>@Test(*timeout = 1000*)</code>	<code>@Test(*timeout = 1000*)</code>
Execute before all tests in this suite have run	N/A	<code>@BeforeSuite</code>
Execute after all tests in this suite have run.	N/A	<code>@AfterSuite</code>
Execute before the test	N/A	<code>@BeforeTest</code>
Execute after the test	N/A	<code>@AfterTest</code>
Execute before the first test method that belongs to any of these groups is invoked.	N/A	<code>@BeforeGroups</code>
Execute after the last test method that belongs to any of these groups is invoked.	N/A	<code>@AfterGroups</code>

Note: In JUnit 4, we declare the `@BeforeClass` and `@AfterClass` methods as static methods. Whereas, in TestNG, there is no such constraint*.

Find Element

- We need to locate the element for interaction with the web element first. And for that, we use **findelement** or **findelements**. Find Element command is used to uniquely identify a (one) web element within the web page. Whereas, the Find Elements command is used to uniquely identify the list of web elements within the web page. You can find the element in multiple ways such as ID, Name, Class Name, Link Text, Partial Link Text, Tag Name, and XPATH.



[Open in app](#)[Get started](#)

```
div.vodal div.vodal-dialog div.modal - dialog div.modal - content  
div.modal - body.text-center div.modal-with-image-text-button  
div.content > h2:nth-child(1)");
```

```
WebElement profile = driver.findElement(By.id("Path_6"));
```

For details about find element & find elements you can visit this [link](#).

Wait (Types of the wait)

- To interact in a web panel, sometimes we need to wait for all the elements for visible. When we click a button, it will redirect us to a new page. For redirecting to this new page, we need some time to interact on the new page. So in this case, we can use wait() or Thread(). But for the performance issue, we will use wait instead of Thread. Because Thread will be off the interaction for the given time. But the wait will be off the interaction until getting the given condition.
- There are types of wait. Those are:

Implicit Wait: Implicit Wait directs the Selenium WebDriver to wait for a certain measure of time before throwing an exception. Once this time is set, WebDriver will wait for the element before the exception occurs.

```
driver.manage().timeouts().implicitlyWait(20, TimeUnit.SECONDS);
```

Explicit Wait: By using the Explicit Wait command, the WebDriver is directed to wait until a certain condition occurs before proceeding with executing the code.

```
WebDriverWait wait = new WebDriverWait(driver, 20);
```

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//*  
[@id=\"signup - page\"]/div[2]/div[1]/h1")));
```



[Open in app](#)[Get started](#)

```
//Declare and initialize a fluent wait

FluentWait wait = new FluentWait(driver);

//Specify the timeout of the wait

wait.withTimeout(5000, TimeUnit.MILLISECONDS);

//Specify polling time

wait.pollingEvery(250, TimeUnit.MILLISECONDS);

//Specify what exceptions to ignore

wait.ignoring(NoSuchElementException.class)

//This is how we specify the condition to wait on.

wait.until(ExpectedConditions.alertIsPresent());
```

- To know more about the wait you can visit the [link](#).

Data Set

- For my project, I need a huge data set. I made a data set with excel with the appropriate data. After that, I converted that file into CSV.

CSVRead: In my script, there are a lot of fields to fill up with the data. There is a sign-up form, sign-in form, and idea submission form. So I don't have the option to provide all the data manually. I need to read the data from the CSV. I had to import Java opencsv.

```
import com.opencsv.CSVReader;

public class csvRead {

    CSVReader signup_data_read;{
```

try {



[Open in app](#)[Get started](#)

```
e.printStackTrace();}}  
  
String[] csvCell;}
```

Assertions

- Assertions are like validations of a test case. For the test cases, we need to check whether the test case is successful or failed. To do this we use Assertions. There are two types of assertions.
- **Hard Assertions:**

Hard Assertions are ones in which test execution is aborted if the test does not meet the assertion condition. Test cases will be aborted for that one condition. Few example of hard assertions:

AssertEquals()

AssertNotEquals()

AssertTrue()

AssertFalse()

AssertNull()

AssertNotNull()

```
Assertions.assertEquals("http://crowdtank.audacityit.work/",  
driver.getCurrentUrl());
```

- **Soft Assertions (Verify Method):**

If the tester wants the script to be not terminated, he can use **Soft Assertions or Verify**



[Open in app](#)[Get started](#)

Hard vs Soft Asserts in Selenium

Hard Assertions	Soft Assertions
Test Execution will be aborted if assert condition is not met	Test execution will continue till the end of the test case even if assert condition is not met
Does not have to invoke any methods to capture the assertions	To view assertions result at the end of the test, tester has to invoke <code>assertAll()</code>

Difference between Assert and Verify in selenium



72



2

- In the case of assertions, if the assert condition is not met, test case execution will be aborted. The remaining tests are skipped, and the test case is marked as failed. These assertions are used as checkpoints for testing or validating business-critical transactions.
- In case of verify, tests will continue to run until the last test is executed even if assert conditions are not met. Verify or Soft Asserts will report the errors at the end of the test. Simply put, tests will not be aborted if any condition is not met. Testers need to invoke the `assertAll()` method to view the results.

Both Hard and Soft Assertions are very important for designing and running [Selenium webdriver](#) tests. They are instrumental in verifying application behavior at critical stages. By using assertions, testing teams can determine if an application is working as it is expected to. They can also save teams the trouble of running tests that don't need to be run if a condition is not met.

Get the Medium app
Selenium IDE



- Selenium IDE (Integrated Development Environment) is a browser extension. For the





Open in app

Get started

record.

- Selenium IDE link for Google Chrome: **Chrome Extension**

